MongoDB Query Optimization Report

Why Indexes Improve Performance

Without an index, MongoDB has no shortcut to locate matching documents: it must read every document in the collection and evaluate the query filter against each one. This is a collection scan (COLLSCAN). An index is a separate, ordered data structure (a B-tree) built on one or more fields; it lets the database jump directly to the range of documents that can match the filter, instead of examining the whole collection. The cost of this improvement is that indexes must be maintained on every write (insert, update, delete), which adds a small overhead and uses extra disk space, so indexes are a trade-off between read speed and write cost, not free performance.

COLLSCAN vs IXSCAN

COLLSCAN (Collection Scan) means the query engine walks through every document in the collection in storage order and tests each one against the filter. Its cost grows linearly with collection size, regardless of how many documents actually match. IXSCAN (Index Scan) means the query engine walks the index's B-tree structure, using the indexed field values to seek directly to the matching key range, and only fetches the full documents that correspond to matching index entries. In the winning plan below, this is visible as an IXSCAN stage feeding into a FETCH stage, where FETCH is the step that retrieves the actual document once the index has identified its location.

Execution Statistics: Before vs After Indexing

Metric	Before Index (COLLSCAN)	After Index (IXSCAN)
Winning plan stage	COLLSCAN	IXSCAN -> FETCH
Index used	none	category_1_published_year_1
totalDocsExamined	34	1
totalKeysExamined	0	1
nReturned	1	1
executionTimeMillis	2 ms	15 ms

The compound index { category: 1, published_year: 1 } was created to match both fields used in the filter. After creating it, the winning plan switched from COLLSCAN to IXSCAN, and totalDocsExamined dropped from 34 (the full collection) to 1 (only the matching document). This is the core evidence that the index is being used correctly: the query no longer needs to inspect documents that cannot match.

Did Execution Time Improve?

Execution Time actually went from 2 ms (COLLSCAN) to 15 ms (IXSCAN). On a collection of only 34 documents, a full collection scan is already extremely cheap, so the fixed overhead of the first use of a newly created index outweighs any savings. This is expected and does not indicate the index is ineffective. The correct signal for index effectiveness is the number of documents examined. In short, execution time is an unreliable indicator on small test collections; totalDocsExamined is the metric that actually demonstrates the index's benefit here.

Conclusion

The compound index on { category, published_year } eliminated 33 of 34 unnecessary document reads for this query by allowing MongoDB to switch from a full COLLSCAN to a targeted IXSCAN. While execution time did not improve on this small dataset, the reduction in documents examined is the metric that predicts real performance gains as the collection grows.